

Task Description for Exercise

Bernhard K. Aichernig, Felix Wallner

Winter Term 2025

1 Model-Based Testing of a Car Alarm System

1.1 Requirements

The topic of the exercise is a controller of a car alarm system with the following requirements:

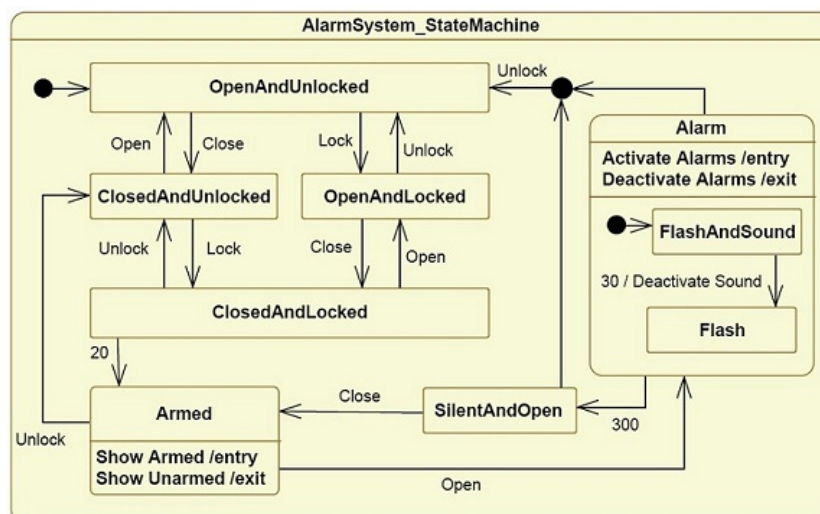
Requirement 1 Arming. The system is armed 20 seconds after the vehicle is locked and the bonnet, luggage compartment, and all doors are closed.

Requirement 2 Alarm. The alarm sounds for 30 seconds if an unauthorised person opens the door, the luggage compartment, or the bonnet. The hazard flasher lights will flash for five minutes.

Requirement 3 Deactivation. The anti-theft alarm system can be deactivated at any time, even when the alarm is sounding, by unlocking the vehicle from outside.

1.2 State Machine

For a better understanding, we provide a state machine according to the requirements:



1.3 Extension with Pincode

The requirements above shall be extended as follows:

Requirement 4: For unlocking, the car key sends a four-digit pincode. This pin is compared to an internally stored code. If the sent code fails to match for three times, then the alarm is triggered. This only happens when the car alarm system is armed.

Requirement 5: It shall be possible to set a new pincode (*setPinCode*). Here, the car must be unlocked and the old and new pincode must be provided. Again, the provision of a wrong pincode for three times triggers the alarm. Otherwise, the system acknowledges the successful update with the message *newPinSet*.

1.4 Refinement of the Doors

Requirement 6: There are four doors plus the bonnet and the luggage compartment. The doors are considered to be closed, when all four doors are closed.

Requirement 7: The luggage compartment can be unlocked separately. Here, the car alarm system will not be deactivated.

1.5 Modelling

Model the car alarm system with the corresponding tool. If necessary, refine the requirements.

1.6 Implementation

Implement the car alarm system in Java, Python, or C# with the corresponding timing behaviour.

1.7 Testing

Test your implementation with the model-based testing tool.

1.8 Mutation Analysis

Use a source code mutation tool in order to analyse the quality of the generated test cases. For Java we recommend PIT¹, for Python there is, e.g., mutmut² and for C# there is, e.g., Stryker³. Report the achieved mutation score.

¹<https://pitest.org>

²<https://pypi.org/project/mutmut/>

³<https://stryker-mutator.io/>

2 Tool Selection

Each group will be assigned one (1) tool from the list below during the exercise assignment meeting, which they will use for the exercise in Section 1.

- **AALpy** - Python, Automata Learning Library
 - <https://github.com/DES-Lab/AALpy>
- **LearnLib** - Java, Automata Learning Library
 - <https://github.com/LearnLib/learnlib>
- **MoMuT (OOAS)** - Model-Based Mutation-Driven Tester, Object Oriented Action System (OOAS) Modelling Language, Test Case Generation
 - <https://momut.org/>
- **Uppaal** - Real-Time System Modelling and Model Checker, Timed-Automata
 - <https://uppaal.org/>
- **Modbat** - Scala/Java, Model-Based Tester
 - <https://gitlab.com/cartho/modbat>
- **TorXakis** - Model-Based Tester, TorXakis Modelling Language, Test Case Generation
 - <https://github.com/TorXakis/TorXakis>
- **Scalacheck** - Scala/Java, Property-Based Testing Library
 - <https://scalacheck.org/>
- **Hypothesis** - Python, Property-Based Testing Library
 - <https://hypothesis.works/>